

Alur training: **Normalisasi input (pixel/255)** → **Susun model: Input(shape) → layer → softmax** → **compile: loss + optimizer + metrik** → **fit dengan validation_split, pantau kurva train vs validation** → **Pilih arsitektur/epoch di validation** → **Uji sekali di data test**

① Cara neural network belajar

- Neuron = kombinasi linear (bobot-masukan + bias) lalu aktivasi; layer = kumpulan neuron
- Loss menyatakan tujuan yang diminimalkan; gradient descent memperbarui bobot berdasarkan arah negatif gradient
- Learning rate mengatur skala pembaruan bobot: terlalu kecil memperlambat training, terlalu besar membuatnya tidak stabil (loss NaN)
- Backprop menghitung gradient loss terhadap tiap bobot, mulai dari output dan menelusuri perhitungan ke layer sebelumnya
- Epoch: satu putaran data train. Batch: kelompok sampel yang diproses bersama; di sini satu batch untuk satu pembaruan bobot

```
model =
tf.keras.Sequential([tf.keras.Input(shape=(28,
28)), Flatten(), Dense(128, 'relu'), Dense(10,
'softmax')])
model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=
['accuracy'])
```

► Mulai dari model kecil, baseline dulu (kelas mayoritas / linear)

② Overfitting & evaluasi

- Loss train turun terus tapi loss validation naik = overfitting; gap yang membesar adalah tandanya
- Obat: dropout (matikan neuron acak saat training, semua aktif saat prediksi), early stopping, data lebih banyak/augmentasi
- Pilih epoch/arsitektur di validation; data test disentuh sekali di akhir
- Confusion matrix menunjukkan kelas mana yang tertukar; akurasi total menyembunyikannya

```
history = model.fit(X_tr, y_tr, epochs=10,
validation_split=0.1)
callbacks=[EarlyStopping(patience=3,
restore_best_weights=True)]
```

► Plot history.history['loss'] vs [val_loss] setiap kali melatih

③ Aktivasi

- Tanpa aktivasi, tumpukan layer tetap linear
- Sigmoid/tanh: turunannya kecil → vanishing gradient di jaringan dalam
- ReLU: default untuk hidden layer; Leaky ReLU kalau banyak neuron mati
- Output: sigmoid untuk biner, softmax untuk multi-kelas (jumlah probabilitas = 1), linear untuk regresi

► Pilihan aman: ReLU di hidden, softmax di output

④ Optimizer

- SGD: langkah tetap searah gradient; momentum menghaluskan arah
- Adam: besar langkah menyesuaikan tiap parameter dari riwayat gradient; pilihan awal yang andal, bukan jaminan terbaik
- Learning rate tetap hyperparameter terpenting, apa pun optimizernya

```
tf.keras.optimizers.Adam(learning_rate=1e-3)
```

► Bandingkan optimizer di validation, bukan test

⑤ CNN

- Konvolusi: filter kecil (3×3) digeser ke seluruh gambar; bobot filter yang sama dipakai di tiap posisi → parameter hemat, pola lokal dicari di seluruh gambar
- Feature map = respons satu filter (tepi, tekstur) per posisi
- Pooling mengecilkan feature map dan tahan pergeseran kecil
- Augmentasi sebagai layer (RandomFlip/RandomRotation/RandomZoom): aktif saat training saja

```
Conv2D(32, 3, activation='relu', padding='same')
→ MaxPooling2D() → Flatten() → Dense
```

► Data gambar; pola spasial lokal

⑥ Transfer learning

- Pakai backbone pre-trained (ResNet50 ImageNet), bekukan bobotnya, latih classification head dengan datamu
- Cocok saat data sedikit dan domain mirip; resolusi jauh dari 224×224 (mis. CIFAR 32×32) membuat feature pre-trained kurang relevan
- Fine-tune sebagian layer atas kalau data cukup, dengan learning rate kecil

```
base = ResNet50(weights='imagenet',
include_top=False, input_shape=(32,32,3));
base.trainable = False
```

► Ribuan gambar, bukan jutaan

⑦ RNN, LSTM, GRU

- Hidden state = memori ringkas yang dibawa antar langkah waktu
- SimpleRNN lupa pada urutan panjang (vanishing gradient); gerbang LSTM/GRU menjaga info lebih lama
- Embedding: indeks kata → vektor padat yang dipelajari
- Baseline deret waktu: nilai berikutnya = nilai terakhir

```
Sequential([Input(shape=(maxlen,)),
Embedding(vocab, 64), LSTM(64), Dense(1,
'sigmoid')])
```

► Teks, sensor, deret waktu; urutan penting

⑧ GPU untuk deep learning

- Banyak operasi training, termasuk perkalian matriks, bisa dijalankan paralel di ribuan core GPU
- Model kecil/batch kecil: overhead pindah data bisa membuat GPU tidak lebih cepat; ukur, jangan asumsi
- Mixed precision memakai FP16 dan FP32 untuk bagian perhitungan yang berbeda; bisa mempercepat dan menghemat memori pada workload yang sesuai — ukur hasilnya. Di nb05, layer output disetel ke float32
- Angka speedup hanya dari pengukuran dengan syaratnya (model, batch, perangkat, setelah warm-up)

```
tf.keras.mixed_precision.set_global_policy('mixed_float16')
```

► Cek 'tf.config.list_physical_devices("GPU")' dulu

⑨ Dari training ke inferensi: TensorRT

- Model final → ONNX (tf2onnx ≥ 1.17) → engine TensorRT (10.x) dengan FP16 untuk latensi rendah di GPU NVIDIA
- TF-TRT sudah tidak ada di TF 2.18+; jalur ONNX yang dipakai
- Bukan alat training; layak saat model melayani banyak permintaan

```
model.export('saved'); python -m tf2onnx.convert
--saved-model saved --output m.onnx --opset 17
```

► Modul 6 membahas deploy lebih jauh

⑩ Model generatif

- Diskriminatif memetakan masukan → label; generatif mempelajari sebaran data dan membuat contoh baru
- Autoencoder: encoder → latent space (ruang ringkas) → decoder; titik berdekatan = gambar mirip
- GAN: generator vs discriminator; kegagalan khas: mode collapse (keluaran seragam)
- Diffusion: belajar membalik noise bertahap; hasil terbaik saat ini pada gambar, tetapi training mahal dan inferensinya bertahap

► Jembatan ke LLM di Modul 4: token menggantikan pixel

Kapan pakai arsitektur yang mana

Data	Arsitektur	Layer kunci	Catatan
Tabel / vektor feature	Dense (MLP)	Dense + ReLU	Normalisasi input; baseline linear dulu
Gambar	CNN / transfer learning	Conv2D, MaxPooling2D, augmentasi	Data sedikit → backbone pre-trained
Teks / urutan	LSTM / GRU (dan Transformer di Modul 3–4)	Embedding, LSTM	Urutan panjang → gerbang, bukan SimpleRNN
Deret waktu	RNN / LSTM	window → SimpleRNN/LSTM	Bandingkan dengan baseline nilai terakhir
Ingin data baru	Autoencoder / GAN / diffusion	encoder–decoder, generator–discriminator	Jujur soal waktu & kualitas

Istilah kunci

Learning rate — besar langkah gradient descent; hyperparameter pertama yang dicoba saat loss tidak turun atau meledak.

Backpropagation — cara menghitung andil tiap bobot pada error, dari output ke layer awal, lalu bobot diperbarui.

Validation vs test — validation untuk memilih model/epoch; test hanya untuk angka akhir, sekali.

Dropout — mematikan neuron acak saat training supaya model tidak bergantung pada jalur tertentu; nonaktif saat prediksi.

Vanishing gradient — gradient menyusut lewat banyak layer/langkah waktu; ReLU dan LSTM/GRU meredakannya.

Feature map — keluaran satu filter konvolusi; di layer awal berupa detektor tepi dan tekstur.

Latent space — representasi ringkas di tengah autoencoder; interpolasi di sini menghasilkan transisi halus.

Mixed precision — sebagian komputasi FP16 untuk kecepatan dan memori; loss dan output dijaga float32.